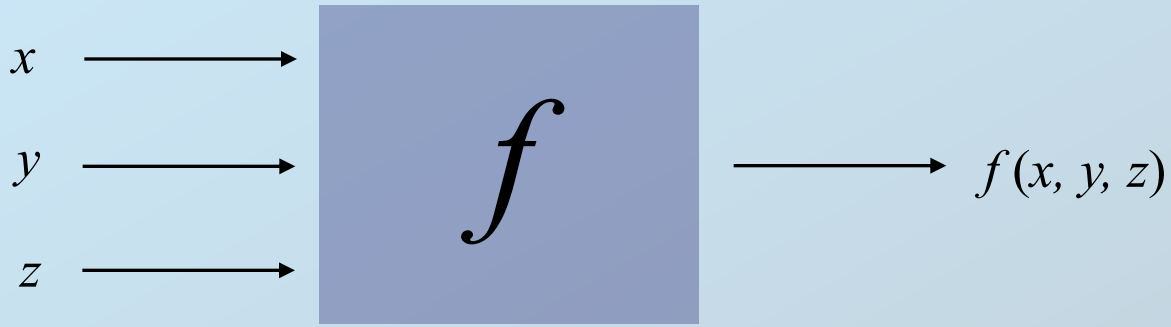
Συναρτήσεις

Αφαιρετικότητα Προγράμματος -
για αρθρωτές λύσεις

Οι συναρτήσεις διευκολύνουν τη ζωή μας και τις έχουμε ήδη αξιοποιήσει σε προηγούμενες Ενότητες του μαθήματος

Συναρτήσεις

- Είναι θεμελιώδεις για τον (δομημένο) προγραμματισμό
- Η χρήση τους άπτεται καίρια των πλείστων από τις αρχές ορθού προγραμματισμού



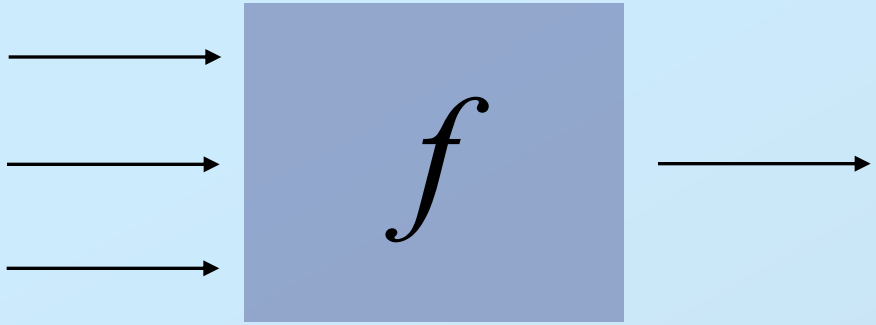
Τι είναι οι Συναρτήσεις;

- Επικοινωνία με το υπόλοιπο πρόγραμμα
  - Δέχονται ορίσματα (έχουν παραμέτρους)
  - Επιστρέφουν μια τιμή
  - Απομονώνουν την επίλυση κάποιου υπό-προβλήματος
- Ιδανικά, δεν θέλουμε καμία άλλη επίδραση στο υπόλοιπο πρόγραμμα πέραν των παραπάνω
  - Καθαρές συναρτήσεις (pure functions), που δεν έχουν καμία παρενέργεια (ή πλευρικό φαινόμενο) πέραν της λειτουργίας που αντικατοπτρίζεται στον τρόπο επικοινωνίας τους (τη διεπαφή/διαπροσωπεία τους) με το υπόλοιπο πρόγραμμα

Σημαντικά Πλεονεκτήματα Συναρτήσεων

- **Πιο αρθρωτός σχεδιασμός προγραμμάτων**
  - Σπάει το πρόγραμμα σε ξεχωριστές εργασίες
  - Κάθε εργασία μπορεί να ανατεθεί σε διαφορετικό προγραμματιστή
- **Επαναχρησιμοποίηση κώδικα**
  - Χρησιμοποίηση του κώδικα ξανά και ξανά
  - Θα μπορούσε να είναι μία Βιβλιοθήκη επαναχρησιμοποιήσιμων μεθόδων (όπως η Βιβλιοθήκη **`Math`**)
  - Λιγότερη επικάλυψη κώδικα
- **Βελτιωμένη αναγνωσιμότητα**
  - Κάθε συνάρτηση μπορεί να κάνει λίγες ξεκάθαρες εργασίες, παρέχοντας μια διαφανή λειτουργικότητα

Γενικά



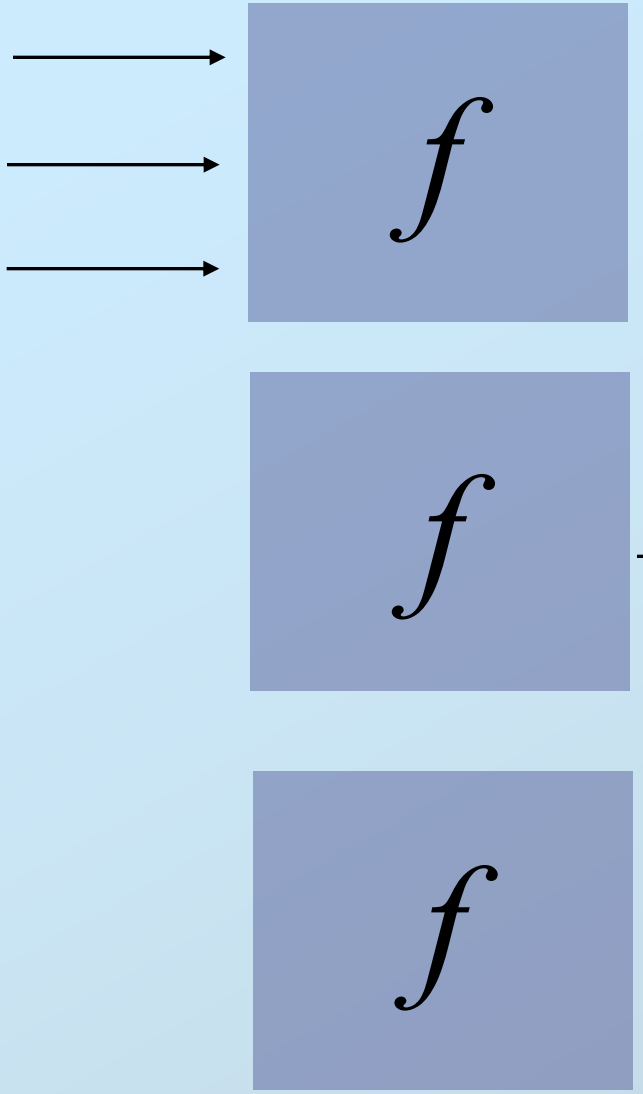
Π.χ. μια συνάρτηση που λαμβάνει ένα πίνακα ακεραίων και επιστρέφει το πλήθος των αρνητικών αριθμών
public static int negative (int [ ] table) {..}

i. *Η συνάρτηση f λαμβάνει τιμές και επιστρέφει τιμή*
ii. *Οι τιμές μπορεί να είναι* ***διευθύνσεις****, π.χ. σε πίνακες ή αντικείμενα*
iii. *Εάν λαμβάνει τιμές που είναι διευθύνσεις, τότε έμμεσα μπορεί να συνιστούν και (επιπρόσθετες) εξόδους*

Διεπαφή Συνάρτησης

- Η διεπαφή/επικοινωνία της με το υπόλοιπο πρόγραμμα (ταυτότητα - signature)
- Ποια είναι η λειτουργία της (τι λαμβάνει, τι επιστρέφει, και επομένως τι λειτουργικότητα παρέχει)
- Αυτή είναι η ουσία (το τι παρέχει δηλαδή), όχι το πώς υλοποιείται η λειτουργικότητα που παρέχει – διαχωρισμός ανάμεσα στο **«τι»** και το **«πώς»**

Πέραν της γενικής διεπαφής με εισόδους και έξοδο μπορεί να έχουμε …



Μόνο εισόδους, οι οποίες δύνανται να είναι διευθύνσεις, π.χ. μια συνάρτηση που ταξινομεί ένα πίνακα
public static void sort (int[] table){..}

Μόνο έξοδο, η οποία επίσης δύναται να είναι διεύθυνση, π.χ. μια συνάρτηση που δημιουργεί ένα πίνακα με τυχαίους αριθμούς από το διάστημα [0,1.0) αξιοποιώντας την Math.random( ) που είναι επίσης αυτής της κατηγορίας
public static double[ ] create ( ){..}

Ούτε εισόδους, ούτε έξοδο, π.χ. μια συνάρτηση που απλά τυπώνει ένα μενού με επιλογές
public static void displayMenu( ){..}

Συναρτήσεις (Στατικές Μέθοδοι)

Java συνάρτηση

- Παίρνει καμία ή περισσότερες παραμέτρους εισόδου
- Επιστρέφει μία τιμή εξόδου – *δεν είναι υποχρεωτικό*

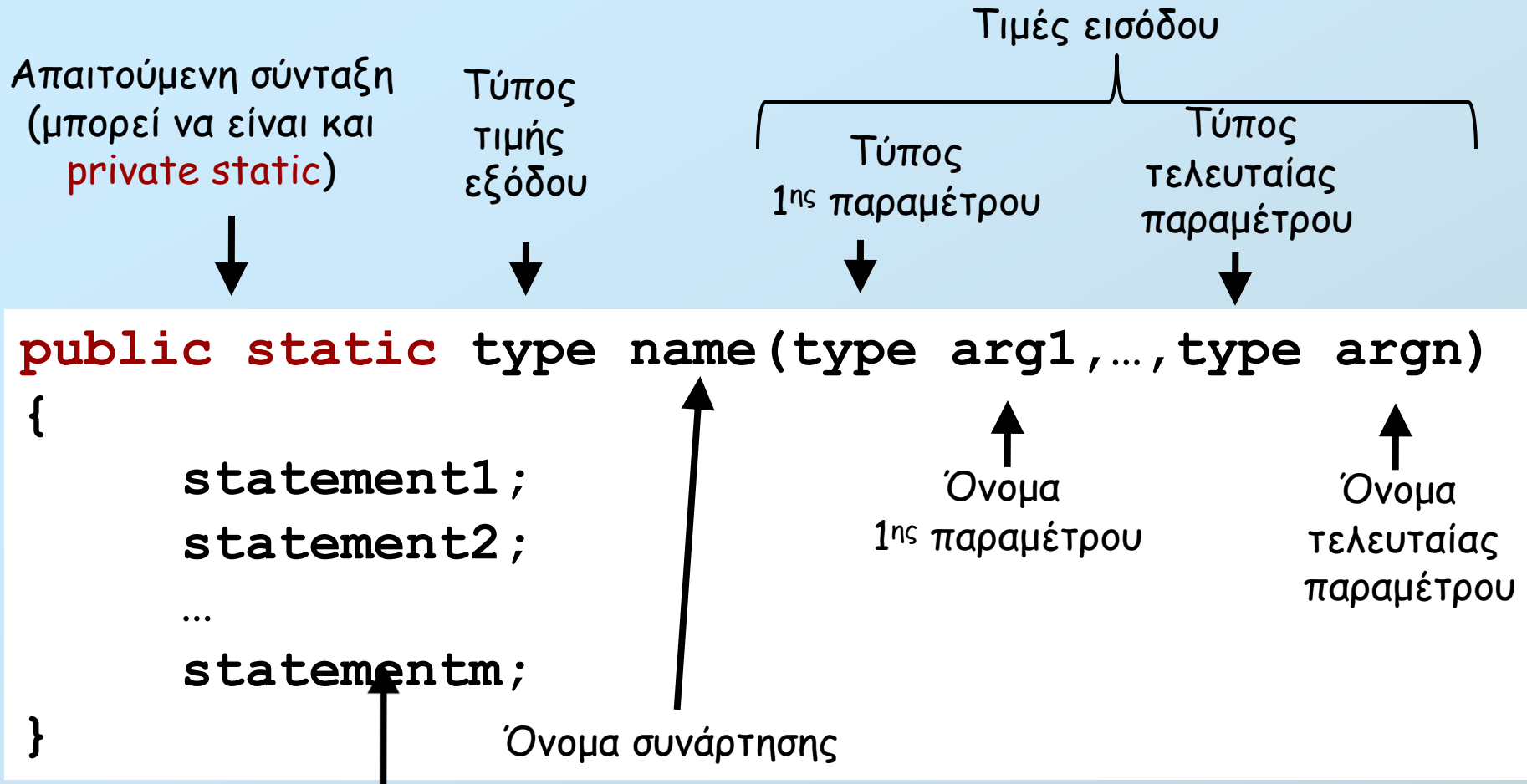
Εφαρμογές

- Οι επιστήμονες χρησιμοποιούν μαθηματικές συναρτήσεις για να υπολογίσουν μαθηματικούς τύπους
- Οι προγραμματιστές χρησιμοποιούν συναρτήσεις για να φτιάξουν **αρθρωτά προγράμματα**
- Εσείς χρησιμοποιείτε συναρτήσεις και για τις δύο παραπάνω εφαρμογές

Παραδείγματα

- Ενσωματωμένες συναρτήσεις: `Math.random()`, `Math.abs()`, `Integer.parseInt()`.
- Οι Βιβλιοθήκες Εισόδου/Εξόδου: `StdIn.readInt()`, `StdDraw.line()`, `StdAudio.play()`.
- Συναρτήσεις ορισμένες από τον χρήστη: `main()`.

Ανατομία της σύνταξης μίας συνάρτησης



Για συναρτήσεις τύπου void μπορεί να κληθεί μόνο χωρίς έκφραση - return;

Συναρτήσεις που έχουν Παραμέτρους

- Παρέχουν μεγαλύτερη αφαιρετικότητα έναντι συναρτήσεων που δεν έχουν παραμέτρους
- Οι παράμετροι ενσωματώνουν τα σημεία που διαφοροποιούν τις κλήσεις μιας συνάρτησης
  - Π.χ. η συνάρτηση `Math.random()` κάνει κάτι το πολύ συγκεκριμένο, που είναι να επιστρέφει ένα αριθμό από το πεδίο [0.0,1.0)
    - Αυτή η λειτουργία δεν μπορεί να διαφοροποιηθεί ανάμεσα στις κλήσεις της συνάρτησης

```
// returns a random integer from interval [a,b]
public static int myRandom (int a, int b){
   return a + (int)(Math.random() * (b-a+1));
}
```

Συναρτήσεις που επιστρέφουν τιμές

- *Οι συναρτήσεις μπορεί να διαχωριστούν:*
  - Σε αυτές που επιστρέφουν μια τιμή, δηλαδή που δίνουν πίσω στο σημείο κλήσης τους μια απάντηση
    - Η απάντηση αναμένεται να τύχει αξιοποίησης, π.χ. να ανατεθεί σε κάποια μεταβλητή, ή να διοχετευτεί ως όρισμα σε κάποια άλλη κλήση συνάρτησης
  - Και σε αυτές που δεν επιστρέφουν πίσω ρητά μια τιμή, είναι δηλαδή void
    - Οι κλήσεις τους αποτελούν standalone εντολές

```
public static int min2 (int a, int b){
  if (a <= b) return a;
  else return b;
}
```

```
int small = min2(x, y);
int smallest = min2(z, min2(x,y));
                  // = min2(z,small);
```

```
public static int min3 (int a, int b, int c){
  return min2(a, min2(b,c));
}
```

```
public static double minT (double[] table) {
  double res = Double.POSITIVE_INFINITY;
  for (double n : table){
    if (n < res) res = n;
  }
  return res;
}
```

```
public static void displayMin (double[] array){
   System.out.println(minT(array))
}
```

```
double[] T = {5.9, 1.0, 7.9,10, 3.1};
displayMin(T); // standalone function call
```

Επισημάνσεις

- Θα ήταν παράλογο συναρτήσεις όπως οι myRandom, min2, min3, και minT, που έχουν ρητές τιμές εξόδου (δίνουν δηλαδή πίσω απαντήσεις) να καλούνται ως standalone εντολές, παρόλο ότι η Java το επιτρέπει
  - Τέτοιες κλήσεις θα δικαιολογούνταν μόνο στην περίπτωση συναρτήσεων που παρέχουν λειτουργικότητα, πέραν του υπολογισμού και της επιστροφής της τιμής εξόδου τους (η οποία απλά αγνοείται), κάτι που δεν ισχύει για τις πιο πάνω συναρτήσεις
    - Διαφορετικά αποτελούν πλεονασμό παραβιάζοντας αρχές ορθού προγραμματισμού

Παράδειγμα συνάρτησης όπου η τιμή εξόδου θα μπορούσε ενδεχομένως να αγνοηθεί

```
// Function swap gets a two-element table xs
// (i.e. the table's address) and if the elements are not
// already in ascending order, they are swapped.
// Through its return value, the function signifies
// whether the swapping was necessary or not.

public static boolean swap (int[] xs){
  if (xs[0] <= xs[1]) return false;
  else {int temp = xs[0]; xs[0] = xs[1]; xs[1] = temp;
     return true; }
}
```

Εντολή return

- Ο κώδικας που αποτελεί την υλοποίηση της λειτουργικότητας που παρέχει μια συνάρτηση (με άλλα λόγια το «πώς» της συνάρτησης) μπορεί να κάνει χρήση όλων των αλγοριθμικών δομών (ακολουθίες, επιλογές, επαναλήψεις, κλήσεις συναρτήσεων, κτλ.) όπως έχουμε ήδη δει στα διάφορα παραδείγματα, π.χ. σε σχέση με τη συνάρτηση main
- Ειδικότερα σε σχέση με την εντολή **return**, όποτε αυτή εκτελείται, η ροή του ελέγχου επιστρέφεται πίσω στο σημείο κλήσης της συνάρτησης
  - Συνάρτηση που έχει τιμή εξόδου πρέπει οπωσδήποτε να περιλαμβάνει τουλάχιστον μια εντολή return μέσω της οποίας επιστρέφεται η τιμή εξόδου (η απάντηση δηλαδή) της κλήσης της συνάρτησης

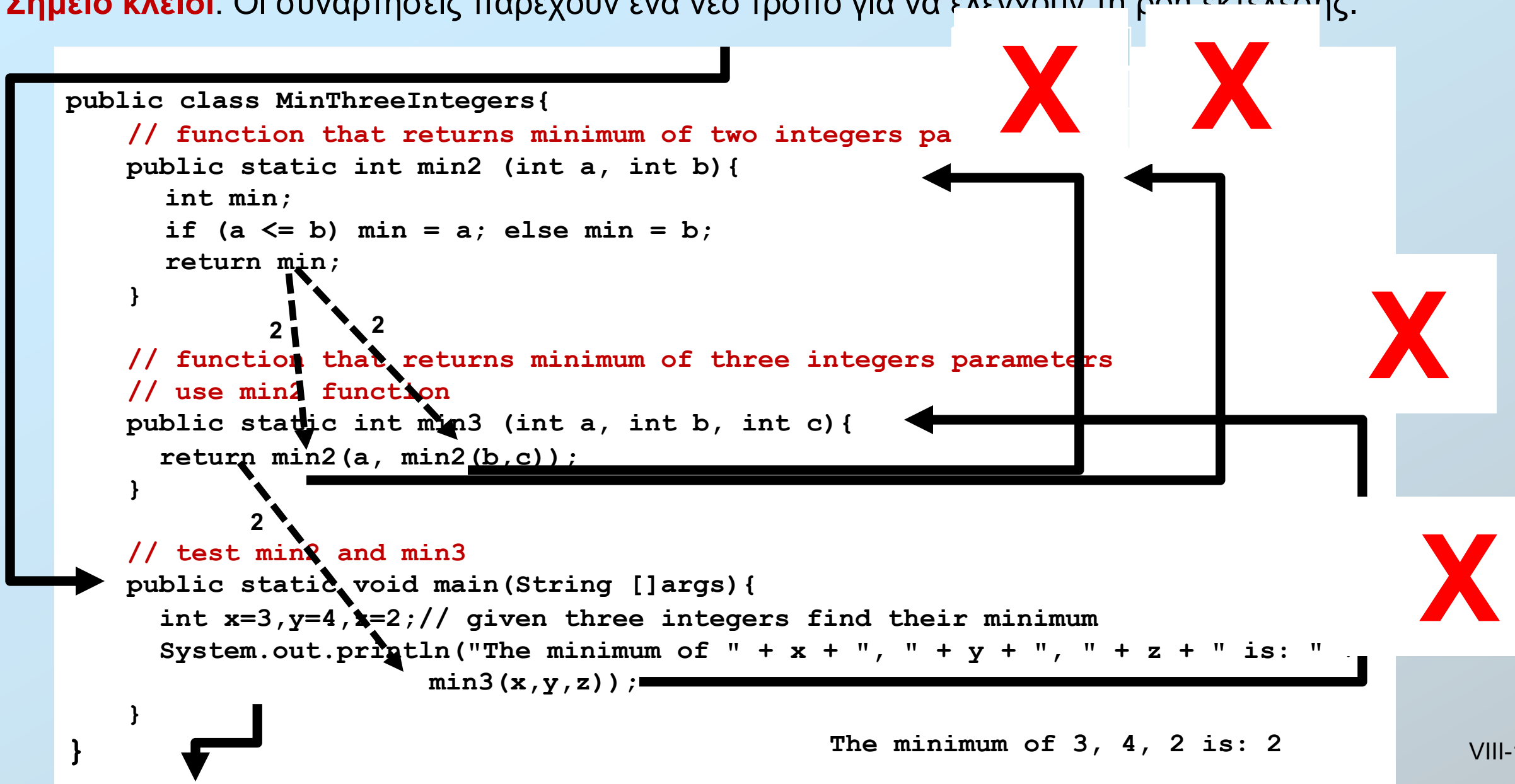
```
return <τιμή εξόδου>
```

  - Συνάρτηση τύπου **void** δύναται επίσης να περιλαμβάνει εντολές return χωρίς αυτές να συνδέονται με τιμές εξόδου, όπου χρειάζεται ο τερματισμός της συνάρτησης (ίσως πρόωρα αλλά όχι αναγκαστικά)

```
return
```

Ροή Ελέγχου κλήση και επιστροφή συνάρτησης

Σημείο κλειδί: Οι συναρτήσεις παρέχουν ένα νέο τρόπο για να ελέγχουν τη ροή εκτέλεσης.



Ροή Ελέγχου

Σύνοψη του τι συμβαίνει όταν μία συνάρτηση καλείται:

- Ο έλεγχος μεταφέρεται στον κώδικα της συνάρτησης
- Στις μεταβλητές παραμέτρων ανατίθενται οι τιμές που δίνονται στην κλήση
- Ο κώδικας της συνάρτησης εκτελείται
- Η επιστρεφόμενη τιμή ανατίθεται στη θέση του ονόματος της συνάρτησης στον κώδικα που καλείται
- Ο έλεγχος επιστρέφει πίσω στον καλούντα κώδικα

Σημ.: Αυτό είναι γνωστό ως «πέρασμα δια τιμής»

```
// Displays a square - returns if size < 1

public static void drawSquare (int size){
  if (size < 1) return;
  for (int i = 1; i <= size; i++){
    System.out.println();
    for (int j = 1; j <= size; j++){
       System.out.print("*");
    }
   }
 }
```

```
drawSquare(0) δεν κάνει τίποτα
```

```
drawSquare(5) δημιουργεί το διακριτό τετράγωνο
*****
*****
*****
*****
*****
```

Καλώντας συναρτήσεις

- Μια συνάρτηση για να αξιοποιηθεί πρέπει να καλεστεί

- Σε κάθε κλήση μίας συνάρτησης δίνονται οι τιμές για τις παραμέτρους της (τα λεγόμενα **ορίσματα** – arguments) και στη συνέχεια αναμένεται η απάντησή της, εάν και εφόσον υπάρχει τιμή εξόδου

```
Class.name(arg1, arg2, arg3);
```

- Εάν η συνάρτηση περιλαμβάνεται στην ίδια κλάση με τον καλούντα κώδικα, τότε δεν χρειάζεται να μπει το όνομα της κλάσης

Ορίσματα (Πραγματικές Παράμετροι)

Arguments (Actual Parameters)

Είναι τιμές (values), οι οποίες μπορεί να διατυπωθούν ως οποιαδήποτε έκφραση

- π.χ. απόλυτη τιμή, μεταβλητή, κλήση συνάρτησης, διεύθυνση (πίνακα/ αντικειμένου) κτλ.

- που αποτιμάται σε τιμή συμβατή με τον τύπο της αντίστοιχης τυπικής παραμέτρου

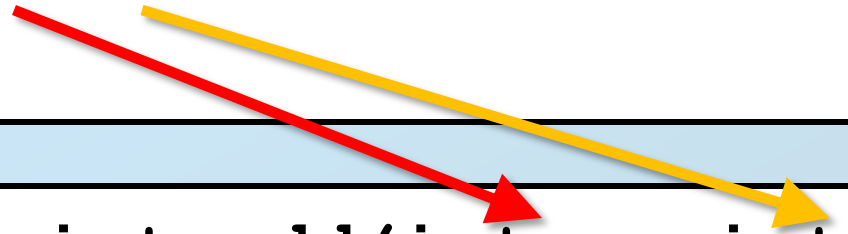
Binding
Δέσμευση

- Μια διαδικασία που ονομάζεται **binding** αντιγράφει τις τιμές (ορίσματα) από τον καλούμενο κώδικα στις παραμέτρους της συνάρτησης
  - Η αντιστοιχία ανάμεσα σε ορίσματα και παραμέτρους είναι μία-προς-μία
- Η κλήση της συνάρτησης μπορεί να χρησιμοποιήσει (ως ορίσματα) μεταβλητές με το ίδιο όνομα με τις παραμέτρους, με διαφορετικά ονόματα, ή ακόμα και απόλυτες τιμές (όπως 3 ή 2.6758)
- Η συνάρτηση δεν αλλάζει τις τιμές των μεταβλητών στον αρχικό κώδικα - Έχει μόνο αντίγραφα των τιμών

Παράδειγμα Binding

```
int a = 10;
int x = 3;
int y = add( 5, a ); //το y περιέχει 15 τώρα
```

```
public static int add(int x, int y){
     int z = x + y; //5 + 10
     return z;
}
```



Σημ.: Δεν υπάρχει καμία σχέση ανάμεσα στα δύο διαφορετικά **x**'s και **y**'s

Αντιστοιχία Ορισμάτων με Παραμέτρους

1. Ο αριθμός ορισμάτων σε δεδομένη κλήση της συνάρτησης πρέπει να είναι ο ίδιος με τον αριθμό παραμέτρων στον ορισμό της συνάρτησης.

2. Η αντιστοιχία αποφασίζεται από τη σειρά των ορισμάτων. Το πρώτο όρισμα αντιστοιχεί στην πρώτη παράμετρο, το δεύτερο όρισμα στη δεύτερη παράμετρο, κτλ.

3. Ο τύπος κάθε ορίσματος πρέπει να είναι συμβατός προς τον τύπο της αντίστοιχης παραμέτρου, δηλαδή ο τύπος μπορεί να καταχωρηθεί στην εν λόγω παράμετρο χωρίς την απροσδόκητη απώλεια πληροφορίας, π.χ. int μπορεί να ανατεθεί σε double χωρίς καθόλου απώλεια πληροφορίας, και double μπορεί να ανατεθεί σε int, με αναμενόμενη όμως την απώλεια των δεκαδικών ψηφίων.

Σημ.: *Η Java επιτρέπει να δοθεί η τιμή int μεταβλητής ως όρισμα σε παράμετρο double, αλλά όχι το ανάποδο*

Εμβέλεια μεταβλητών

- Οι μεταβλητές από έξω από μια συνάρτηση δεν υπάρχουν αν δεν έχουν περαστεί ως παράμετροι

```
public static int add(int x, int y){
      int z = x + y;
      return z;
}
```

- Ανεξάρτητα με το πόσο πολύπλοκο είναι ένα πρόγραμμα, μέσα στη συνάρτηση **add** μόνο οι **x**, **y**, και **z** μεταβλητές υπάρχουν

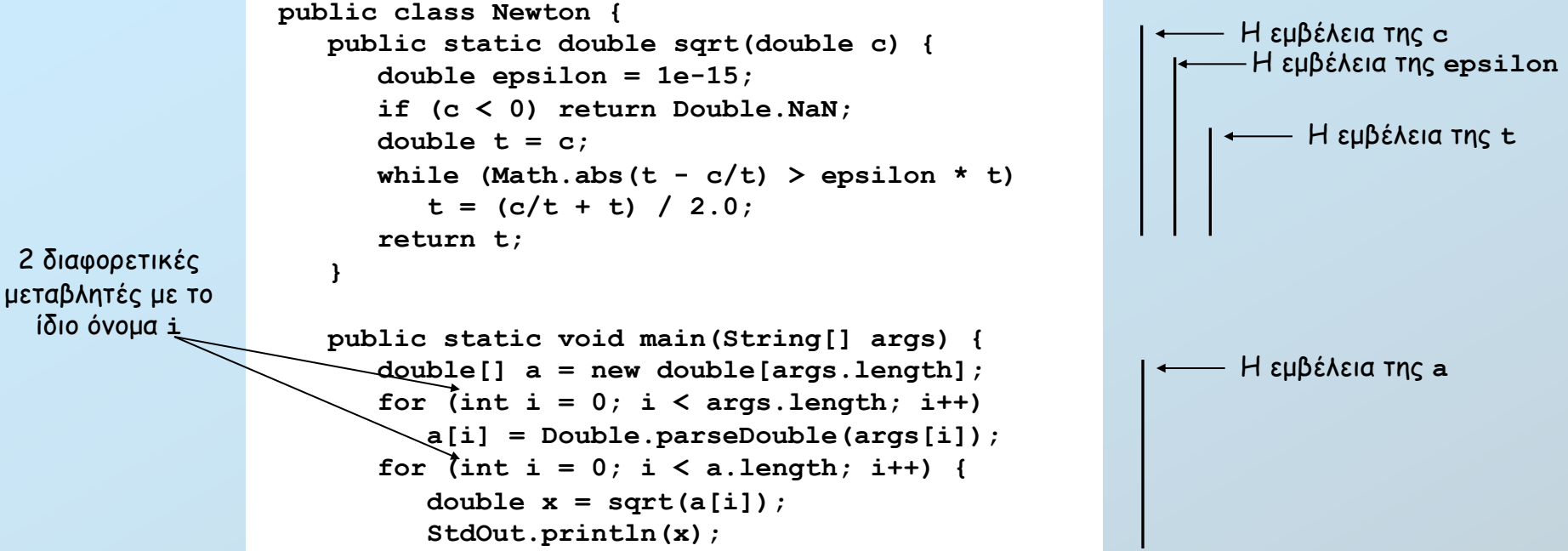
Εμβέλεια μεταβλητών

Εμβέλεια (ενός ονόματος): Ο κώδικας ο οποίος αναφέρεται σε αυτό το όνομα.

Π.χ. Η εμβέλεια μίας μεταβλητής είναι ο κώδικας που ακολουθεί τη δήλωση στο μπλοκ

```
public class Newton {
   public static double sqrt(double c) {
      double epsilon = 1e-15;
      if (c < 0) return Double.NaN;
      double t = c;
      while (Math.abs(t - c/t) > epsilon * t)
         t = (c/t + t) / 2.0;
      return t;
   }

   public static void main(String[] args) {
      double[] a = new double[args.length];
      for (int i = 0; i < args.length; i++)
         a[i] = Double.parseDouble(args[i]);
      for (int i = 0; i < a.length; i++) {
         double x = sqrt(a[i]);
         StdOut.println(x);
      }
   }
}
```



Η καλύτερη πρακτική: δηλώστε μεταβλητές για να περιορίσετε την εμβέλεια τους

Συναρτήσεις χρήστη με παραμέτρους πίνακες

```
//function that returns min value in a table of doubles
public static double minTable (double[] table){
  double res = Double.POSITIVE_INFINITY;
  for (int i=0;i<table.length;++i){
    if (table[i] < res) res = table[i];
  }
  return res;
}
```

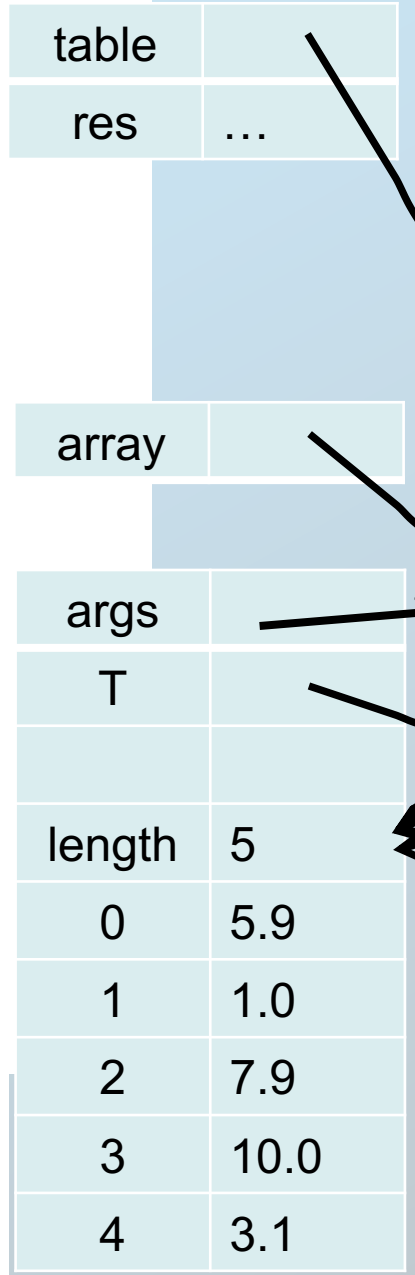
```
// function that prints the min value in a table of doubles
// uses minTable, does not return anything void
public static void displayMinTable(double[] array){
   System.out.println(minTable(array));
}
```

Συναρτήσεις χρήστη με παραμέτρους πίνακες

```
public class MinTableDoubles{

    //function that returns min value in a table of doubles
    public static double minTable (double[] table){
      double res = Double.POSITIVE_INFINITY;
      for (int i=0;i<table.length;++i){
        if (table[i] < res) res = table[i];
      }
      return res;
    }
    // function that prints the min value in a table of doubles
    // uses minTable
    public static void displayMinTable (double[] array){
       System.out.println(minTable(array));
    }

    public static main(String[] args){
       double[] T = {5.9, 1.0, 7.9, 10, 3.1};
       displayMinTable(T); // function call
    }
}
```



- Η main περνάει τον T by reference (αναφορά, pointer, διεύθυνση) στη συνάρτηση displayMinTable
- Ο array στη displayMinTable μετά την κλήση από την main δείχνει στον ίδιο πίνακα όπως η main, δηλ. τον πίνακα T
- Ο table στη minTable μετά από την κλήση από τη δείχνει στον ίδιο πίνακα όπως main, δηλ. τον T

Ειδή παραμέτρων και τύπων επιστροφής συνάρτησης

- Οι τιμές που λαμβάνει (παράμετροι) μπορεί να είναι τιμές έκφρασης (δια τιμής/by value), ή αναφορές (διευθύνσεις, π.χ. σε πίνακες ή αντικείμενα (references)
- Μια συνάρτηση επιστρέφει τιμή ή αναφορά
- Εάν λαμβάνει τιμές που είναι αναφορές, τότε έμμεσα μπορεί να συνιστούν και (επιπρόσθετες) εξόδους

Πίνακας σαν παράμετρο εξόδου

```
public class InitializeTableDoubles{

    //function that initializes a table of doubles
    public static void initializeTable (double[] table, int v){
       for (int i=0;i<table.length;++i)
        table[i] = v;
    }
    // function that display contents of a table of doubles
    public static void displayTable(double[] array){
      for(int i=0;i<array.length;++i) System.out.println(i + " : " + array[i]);
    }

    public static main(String[] args){
       double[] T = new double[Integer.parseInt(args[0])];
       initializeTable(T, Integer.parseInt(args[1])); // function call
       displayTable(T);
    }
}
```

args	
T	

```
java InitializeTableDoubles 3 436
0 : 436
1 : 436
2 : 436
```

Πίνακας σαν παράμετρο εξόδου

```
public class InitializeTableDoubles{

    //function that initializes a table of doubles
    public static void initializeTable (double[] table, int v){
       for (int i=0;i<table.length;++i)
        table[i] = v;
    }
    // function that display contents of a table of doubles
    public static void displayTable(double[] array){
      for(int i=0;i<array.length;++i) System.out.println(i + " : " + array[i]);
    }

    public static main(String[] args){
       double[] T = new double[Integer.parseInt(args[0])];
        initializeTable(T, Integer.parseInt(args[1])); // function call
      displayTable(T);
    }
}
```

args	
T	
length	3
0	0.0
1	0.0
2	0.0

```
java InitializeTableDoubles 3 436
0 : 436
1 : 436
2 : 436
```

Πίνακας σαν παράμετρο εξόδου

```
public class InitializeTableDoubles{

    //function that initializes a table of doubles
    public static void initializeTable (double[] table, int v){
       for (int i=0;i<table.length;++i)
        table[i] = v;
    }
    // function that display contents of a table of doubles
    public static void displayTable(double[] array){
      for(int i=0;i<array.length;++i) System.out.println(i + " : " + array[i]);
    }

    public static main(String[] args){
       double[] T = new double[Integer.parseInt(args[0])];
        initializeTable(T, Integer.parseInt(args[1])); // function call
      displayTable(T);
    }
}
```

table	
v	436

args	
T	
length	3
0	0.0
1	0.0
2	0.0

```
java InitializeTableDoubles 3 436
0 : 436
1 : 436
2 : 436
```

Πίνακας σαν παράμετρο εξόδου

```
public class InitializeTableDoubles{

    //function that initializes a table of doubles
    public static void initializeTable (double[] table, int v){
       for (int i=0;i<table.length;++i)
        table[i] = v;
    }
    // function that display contents of a table of doubles
    public static void displayTable(double[] array){
      for(int i=0;i<array.length;++i) System.out.println(i + " : " + array[i]);
    }

    public static main(String[] args){
       double[] T = new double[Integer.parseInt(args[0])];
        initializeTable(T, Integer.parseInt(args[1])); // function call
      displayTable(T);
    }
}
```

table	
v	436

args	
T	
length	3
0	436
1	436
2	436

```
java InitializeTableDoubles 3 436
0 : 436
1 : 436
2 : 436
```

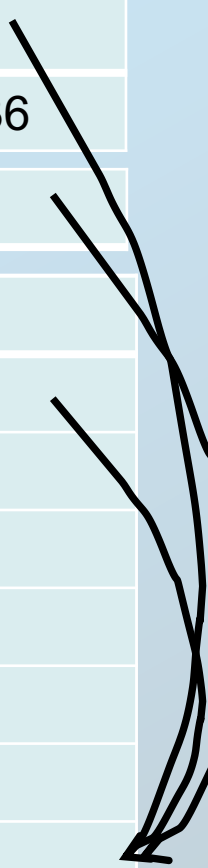
Πίνακας σαν παράμετρο εξόδου

```
public class InitializeTableDoubles{

    //function that initializes a table of doubles
    public static void initializeTable (double[] table, int v){
       for (int i=0;i<table.length;++i)
        table[i] = v;
    }
    // function that display contents of a table of doubles
    public static void displayTable(double[] array){
      for(int i=0;i<array.length;++i) System.out.println(i + " : " + array[i]);
    }

    public static main(String[] args){
       double[] T = new double[Integer.parseInt(args[0])];
        initializeTable(T, Integer.parseInt(args[1])); // function call
      displayTable(T);
    }
}
```

table	
v	436
array	
args	
T	
length	3
0	436
1	436
2	436



```
java InitializeTableDoubles 3 436
0 : 436
1 : 436
2 : 436
```

Διάσπαση και Σύνθεση

Διαίρει και Βασίλευε

Βασική επιδίωξη ακόμη και για απλά προβλήματα είναι η δημιουργία αρθρωτών λύσεων αξιοποιώντας συναρτήσεις

Function Overloading

- Μπορούμε να ορίσουμε εναλλακτικές εκδοχές μιας συνάρτησης οι οποίες διακρίνονται βάσει του signature τους
  - Κάποιες εκδοχές μπορεί να είναι private
  - Το function overloading ικανοποιεί την αρχή του “open for extension” αφού
  - Με ευκολία μπορούν να προστίθενται νέες εκδοχές συναρτήσεων που ήδη αξιοποιούνται χωρίς επηρεασμό της/των υφιστάμενης/ων εκδοχής/ών τους

Το API της Math

τρεις εκδοχές της συνάρτησης min βάση των τύπων των παραμέτρων της

```
public class Math
_____________________________________________________________________
        int    min(int,int)
        float  min(float,float)
        double min(double, double)
```

```
int a,b,x;
float c,d,y;
double e,f,z;
//above variables initalized

x=min(a,b); // calls int
y=min(c,d); // calls float
z=min(e,f); // calls double
z=min(a,f); // calls double - casts automatically a to double
```

```
public class MinThreeIntegers{
    // function that returns minimum of two integers parameters
    public static int min(int a, int b){
       int min;
       if (a <= b) min = a; else min = b;
       return min;
    }

    // function that returns minimum of three integers parameters
    // use min function
    public static int min(int a, int b, int c){
      return min(a, min(b,c));
    }

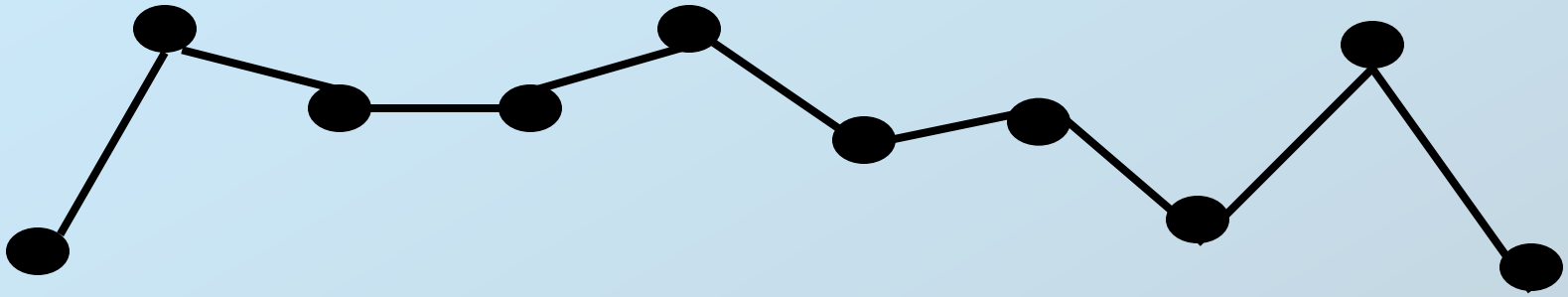
    // test min2 and min3
    public static void main(String []args){
      int x=3,y=4,z=2; // given three integers find their minimum
      System.out.println("The minimum of " + x + ", " + y + ", " + z + " is: " +
                         min(x,y,z));
    }
}
```

Παράδειγμα: Δημιουργία Βιβλιοθήκης DrawGraph για τον σχεδιασμό γραφικών παραστάσεων

Plotting continuous graphs

ys

0	1	2	3	4	5	6	7	8	9



- *Θα αξιοποιηθεί η Βιβλιοθήκη StdDraw* (κλάση του βιβλίου)
- *Θα γίνει χρήση της έννοιας του function overloading – υπερφόρτωση συναρτήσεων*

```
import java.awt.Color; // Uses colors - Color is an object

public class DrawGraph {

  // returns the maximum element of table ys
  private static double max (double[] ys){
   double M = Double.NEGATIVE_INFINITY;
   for (int i = 0; i < ys.length; i++) {
     if (M < ys[i]) M = ys[i];
   }
   return M;
  }

  // returns the minimum element of table ys
  private static double min (double[] ys){
   double M = Double.POSITIVE_INFINITY;
   for (int i = 0; i < ys.length; i++) {
     if (M > ys[i]) M = ys[i];
   }
   return M;
  }
  // Class definition continues . . .
```

Το API της DrawGraph

Δυο εκδοχές της συνάρτησης plot

```
public class DrawGraph
```

`void plot (double[] ys)`	`plots one graph`
`void plot (double[] ys1, double[] ys2)`	`plots two graphs – ys1 and ys2 are parallel arrays`

```
// Continuation of DrawGraph class definition

 public static void plot (double[] ys){

    double MIN = min(ys);
    double MAX = max(ys);
    StdDraw.setXscale(0,ys.length-1);
    StdDraw.setYscale(MIN,MAX);
    StdDraw.setPenRadius(0.004);
    StdDraw.setPenColor(StdDraw.BLACK);
    StdDraw.clear(StdDraw.WHITE);

    for (int i = 0; i < ys.length-1; i++){
      StdDraw.line(i, ys[i], i+1, ys[i+1]);
    }
 }

// class definition continues . . .
```

```
// Continuation of DrawGraph class definition

public static void plot (double[] ys1, double[] ys2){
   double MIN = min(ys1);
   if (MIN > min(ys2)) MIN = min(ys2);
   double MAX = max(ys1);
   if (MAX < max(ys2)) MAX = max(ys2);

   StdDraw.setXscale(0,ys1.length-1);
   StdDraw.setYscale(MIN,MAX);
   StdDraw.clear(StdDraw.WHITE);
   StdDraw.setPenRadius(0.004);

   for (int i = 0; i < ys1.length-1; i++){
     StdDraw.setPenColor(StdDraw.RED);
     StdDraw.line(i, ys1[i], i+1, ys1[i+1]);
     StdDraw.setPenColor(StdDraw.GREEN);
     StdDraw.line(i, ys2[i], i+1, ys2[i+1]);
   }
}
```

```
// How does the highlighted code differ from its
// previous version?

 public static void plot (double[] ys1, double[] ys2){

                . . . . . .

   for (int i = 0; i < ys1.length-1; i++){
     StdDraw.setPenColor(StdDraw.RED);
     StdDraw.line(i, ys1[i], i+1, ys1[i+1]);
   }
   for (int i = 0; i < ys2.length-1; i++){
     StdDraw.setPenColor(StdDraw.GREEN);
     StdDraw.line(i, ys2[i], i+1, ys2[i+1]);
   }
 }
```

Απάντηση …

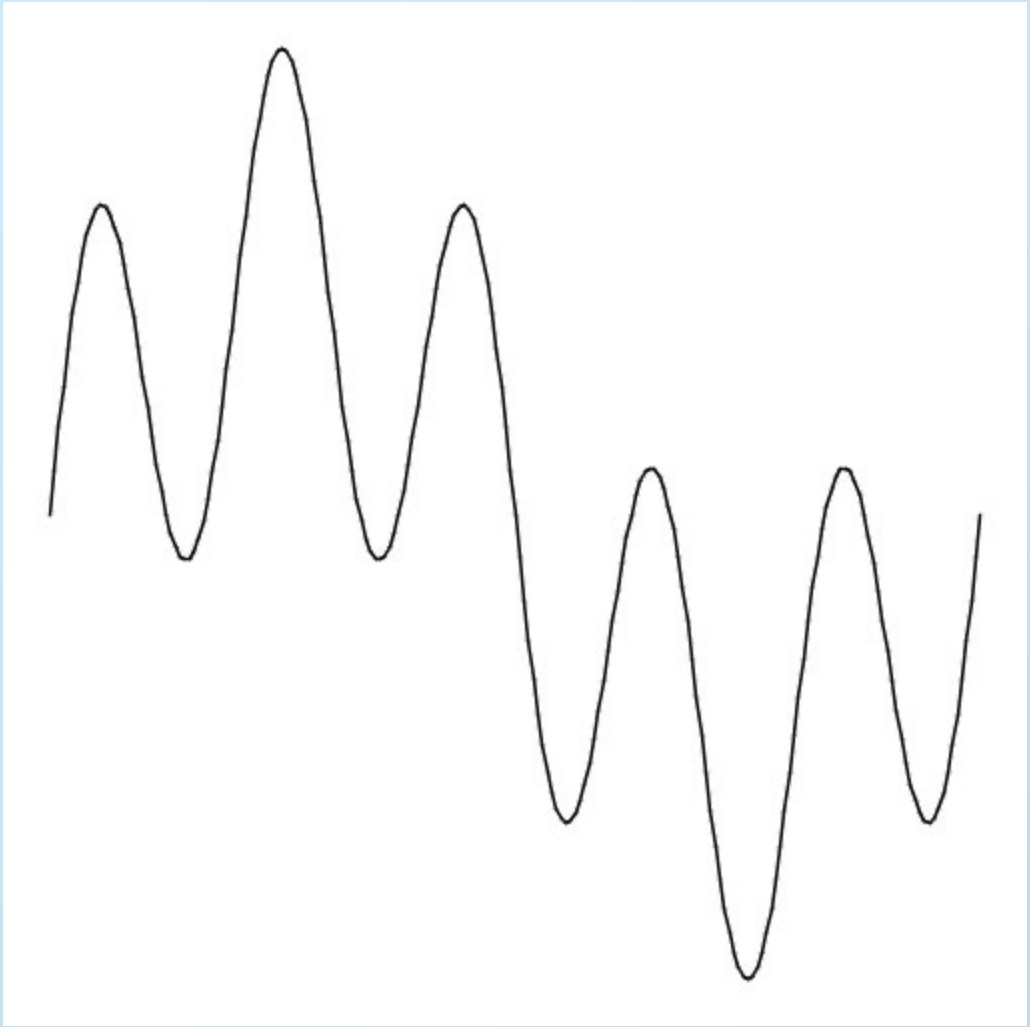
- Πρώτα σχεδιάζεται εξ ολοκλήρου η κόκκινη γραφική παράσταση και μετά η πράσινη – στον προηγούμενο κώδικα οι δύο παραστάσεις σχεδιάζονται ταυτόχρονα
- Επιπρόσθετα, τώρα οι πίνακες ys1 και ys2 δεν χρειάζεται να έχουν το ίδιο πλήθος στοιχείων, αλλά και πάλι, σε περίπτωση διαφορετικού πλήθους, ο μικρότερος πίνακας θεωρείται παράλληλος του αρχικού τμήματος του μεγαλύτερου πίνακα

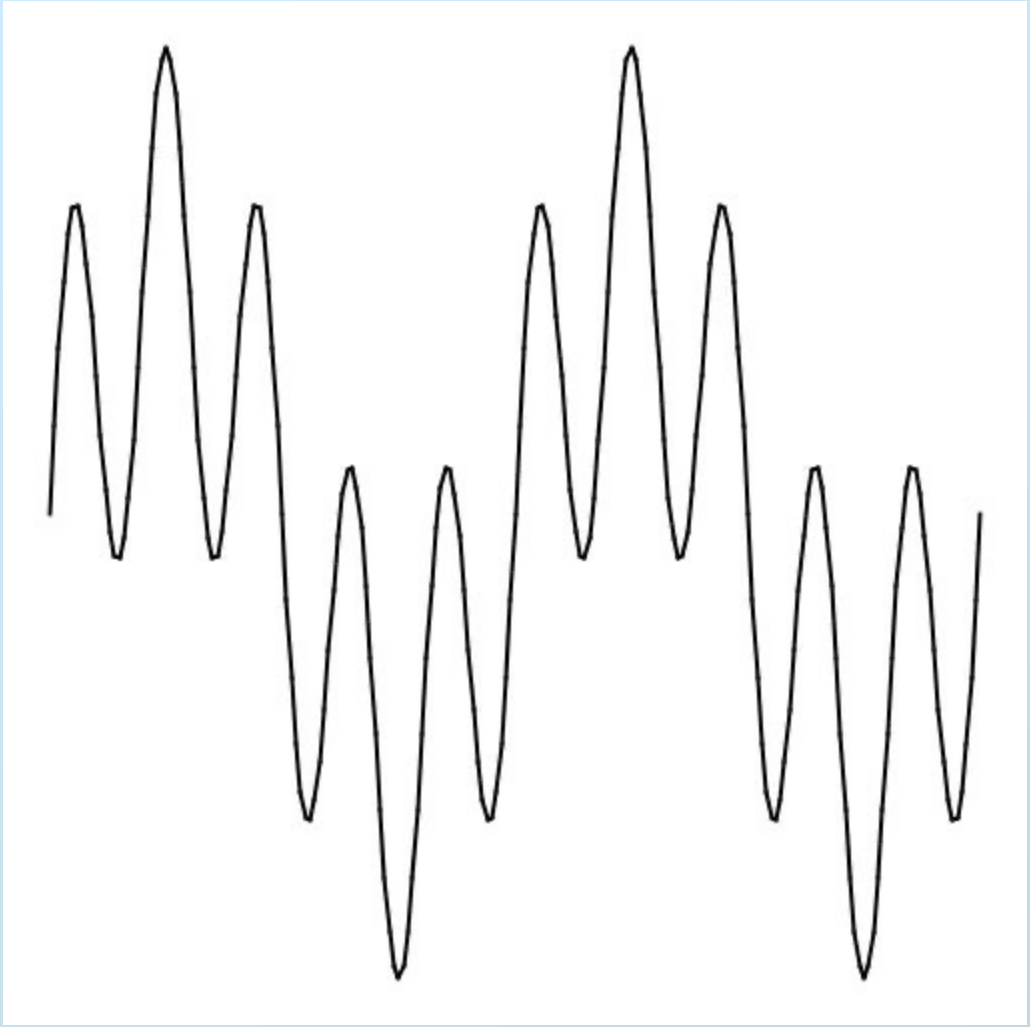
```
// The class definition is completed with function main
// for purely testing purposes

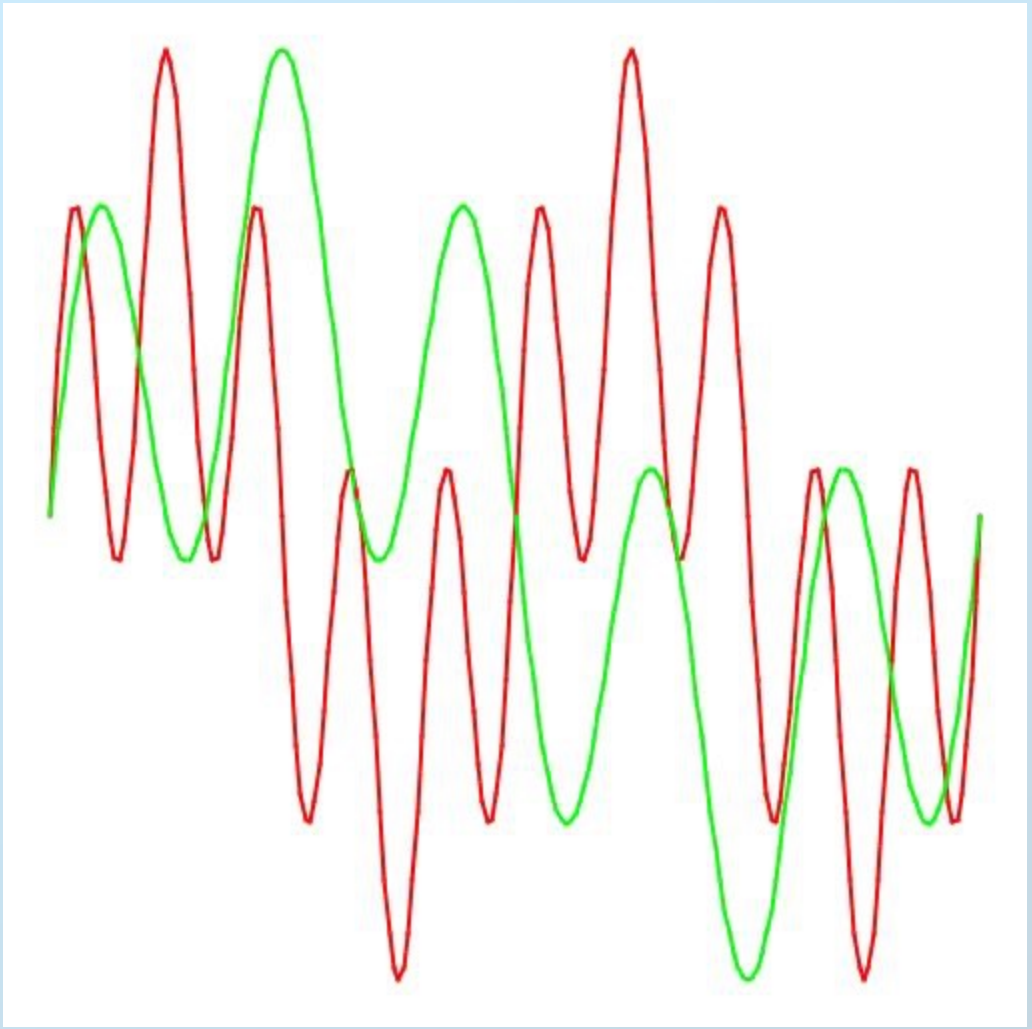
  public static void main (String[] args){
      int N = Integer.parseInt(args[0]);
      double[] ys = new double[N+1];
      double[] zs = new double[N+1];
      for (int x = 0; x < N; x++){
         ys[x] = Math.sin(4*Math.PI*x/N) +
                 Math.sin(20*Math.PI*x/N);
         zs[x] = Math.sin(2*Math.PI*x/N) +
                 Math.sin(10*Math.PI*x/N);
      }
      plot(zs); StdDraw.show(500);
      plot(ys); StdDraw.show(500);
      plot(ys, zs); StdDraw.show(500);
    }

  }
```

```
$ java DrawGraph 200
```







Παράδειγμα: Δημιουργία Βιβλιοθήκης SortLib – *Συναρτήσεις Ταξινόμησης*

- Αρχικά θα περιλαμβάνει συναρτήσεις για την ταξινόμηση πινάκων με ακεραίους, βάσει των γνωστών μεθόδων
  - Bubble sort – ταξινόμηση φυσαλίδας
  - Insertion sort – ταξινόμηση εισαγωγής
- Για σκοπούς δοκιμής θα αξιοποιηθούν βοηθητικές συναρτήσεις καθώς και η Βιβλιοθήκη DrawGraph
  - Παρουσίαση χρόνου εκτέλεσης για διάφορα μεγέθη πινάκων – complexity estimates

Το API της SortLib

```
public class SortLib

_____________________________________________________________________

    void insertionSort (int[] table)  applies insertion sort
    void bubbleSort (int[] table)     applies bubble sort
```

```
public class SortLib {

// Ancillary function for testing purposes - Is table sorted?
private static boolean sorted (int[] table){
   for (int i = 0; i < table.length-1; i++)
     if (table[i] > table[i+1]) return false;
   return true;
  }

 // INSERTION SORT - Complexity: O(n2)
 public static void insertionSort (int[] table){
    for (int i = 1; i < table.length; i++){
      boolean stop = false;
      for (int j = i; !stop && j > 0; j--){
        if (table[j] < table[j-1]){
            int temp = table[j-1];
            table[j-1] = table[j];
            table[j] = temp;
           }
         else stop = true;
      }
    }
  }
```

```
// continuation of SortLib . . .

  // BUBBLE SORT - Complexity: O(n2)
  public static void bubbleSort(int[] table){
    boolean sorted = true;
    int index = table.length;
    do {
        sorted = true; index--;
        for (int i = 0; i < index; i++){
            if (table[i] > table[i+1]){
                sorted = false;
                int temp = table[i];
                table[i] = table[i+1];
                table[i+1] = temp;
              }
        }
     } while(index > 0 && !sorted);
  }

  public static void main(String[] args){. . . . .}

}
```

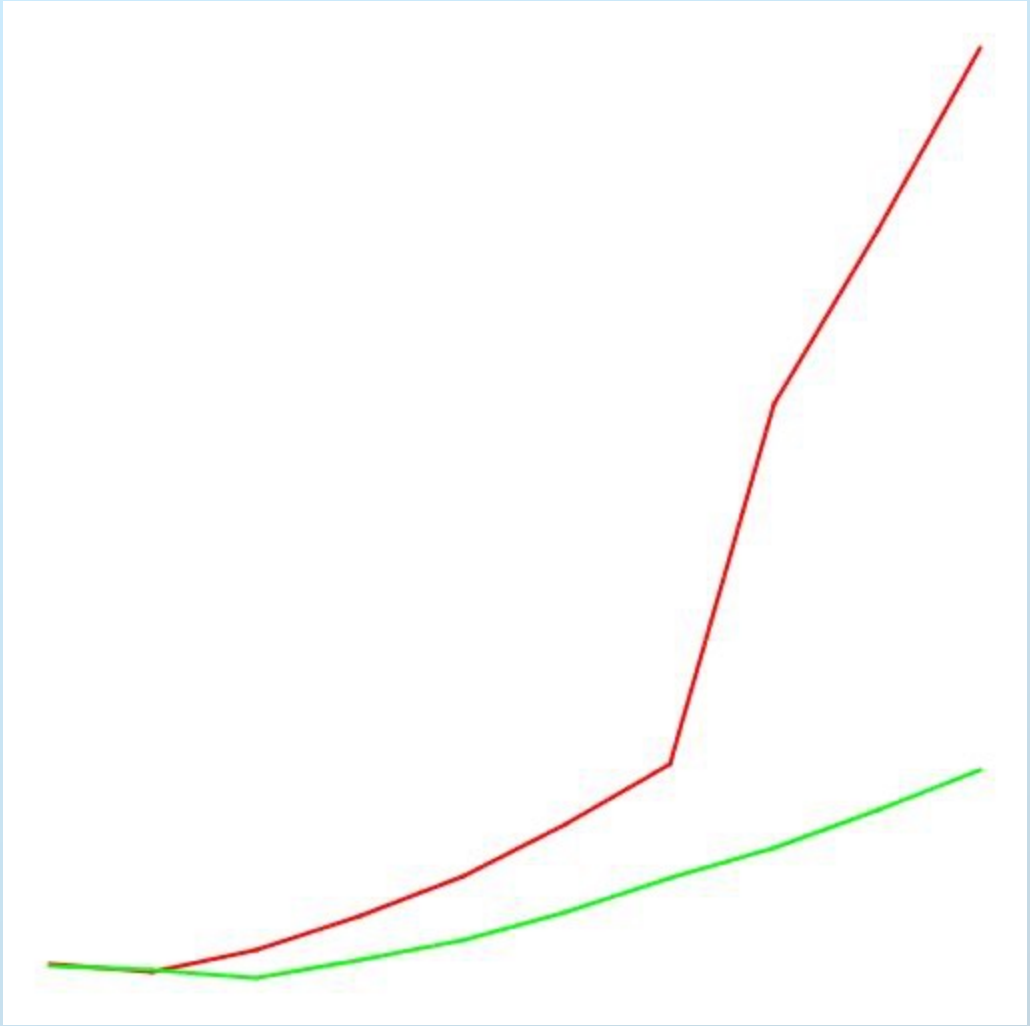
Συμπληρώστε τη main για τη δοκιμή της SortLib

- Κάνετε χρήση της ιδιωτικής, βοηθητικής συνάρτησης sorted για να βεβαιωθείτε ότι οι συναρτήσεις insertionSort και bubbleSort είναι ορθές
  - Σε μεταγενέστερο στάδιο θα σας ζητηθεί να προσθέσετε και άλλες μεθόδους ταξινόμησης (selection sort, merge sort, quick sort)
- Αξιοποιείστε τη μέθοδο **`System.nanoTime()`** για να μετρήσετε το χρόνο εκτέλεσης των συναρτήσεων ταξινόμησης, π.χ.

```
long start = System.nanoTime();
insertionSort(table);
long elapsedTime = System.nanoTime() - start;
                    // measures time in nano seconds
```

- Για κάθε συνάρτηση ταξινόμησης:
  - δημιουργείστε πίνακες με τυχαίο τρόπο (π.χ. να περιλαμβάνουν στοιχεία από το 0 μέχρι το 100) και διαφορετικά πλήθη στοιχείων (π.χ. 100, 200,.., 1000)
  - Για κάθε πλήθος επαναλάβετε για αρκετές φορές (π.χ. 50) και υπολογίστε τον μέσο χρόνο εκτέλεσης
  - Παρουσιάστε τα αποτελέσματα τόσο αριθμητικά, όσο και ως γραφικές παραστάσεις

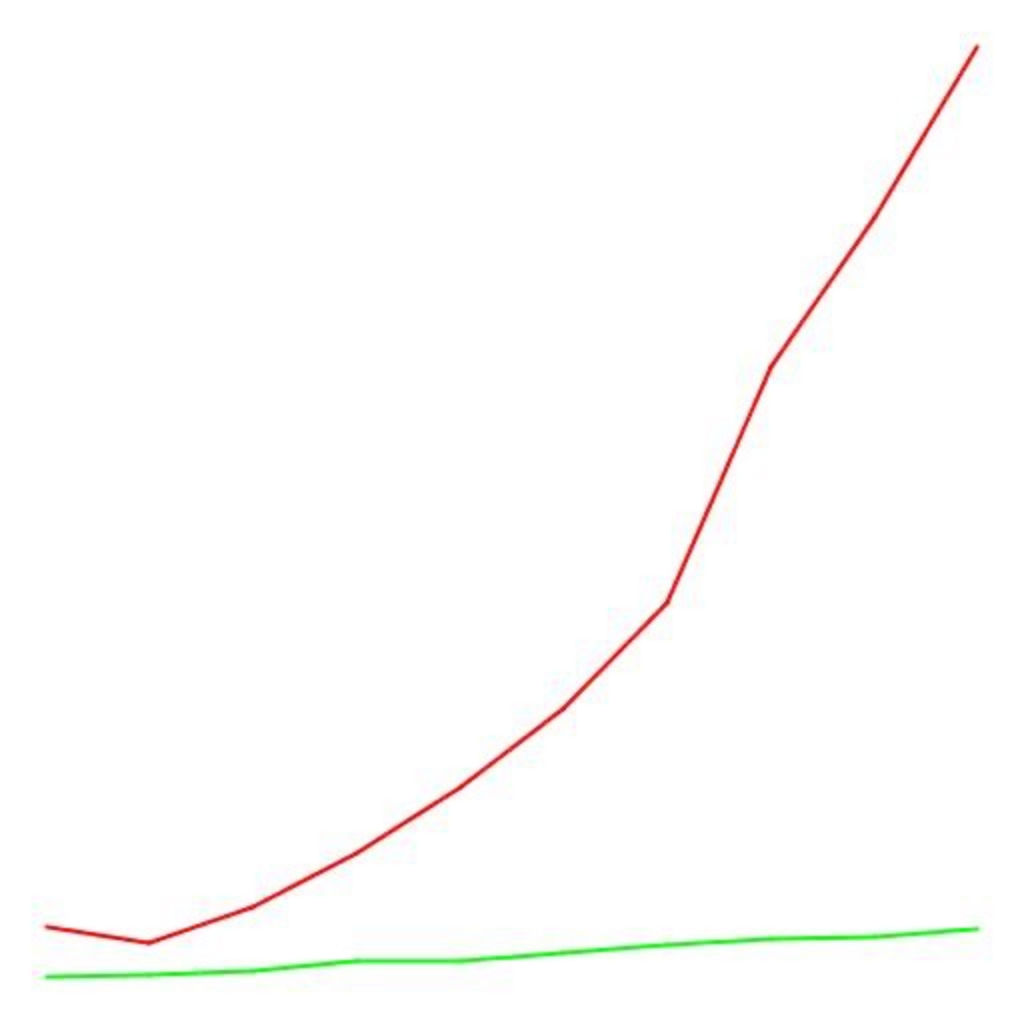
```
$ java SortLib
Size            Bubble Sort             Insertion Sort
100             60806.7                 55677.54
200             47959.0                 51027.04
300             81734.96                37535.62
400             136383.56               66598.16
500             198793.42               98489.1
600             278579.74               140414.6
700             373821.12               193244.74
800             934553.1                242840.02
900             1208062.88              299789.16
1000            1493227.1               364276.86
```



Bubble sort - $O(n^2)$

Insertion sort - $O(n^2)$

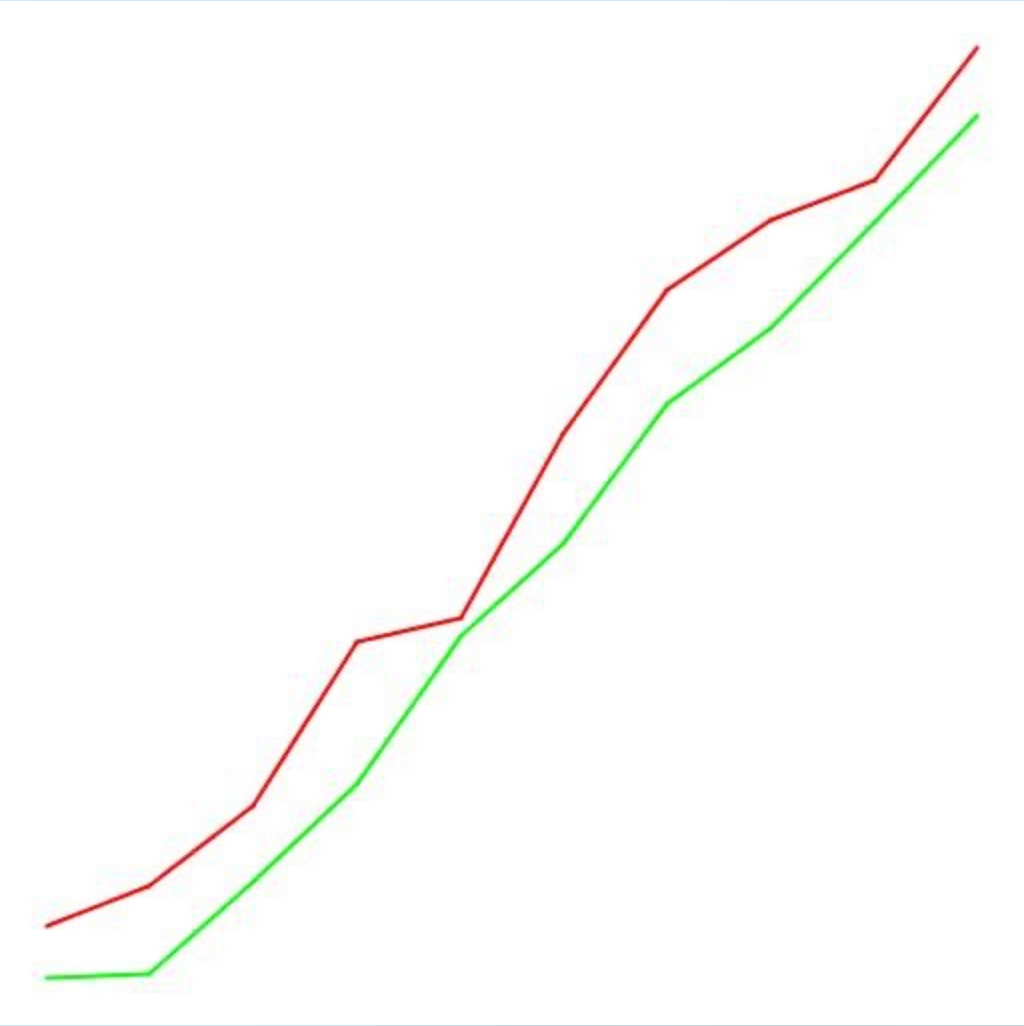
```
$ java SortLib
Size            Bubble Sort             Merge Sort
100             60630.62                12336.56
200             45975.3                 13833.9
300             79319.92                17774.92
400             131352.8                27668.02
500             192955.92               27699.5
600             267379.54               36389.02
700             368356.22               42808.16
800             592907.68               49234.12
900             736253.0                51635.8
1000            897496.1                58596.84
```



Bubble sort - $O(n^2)$

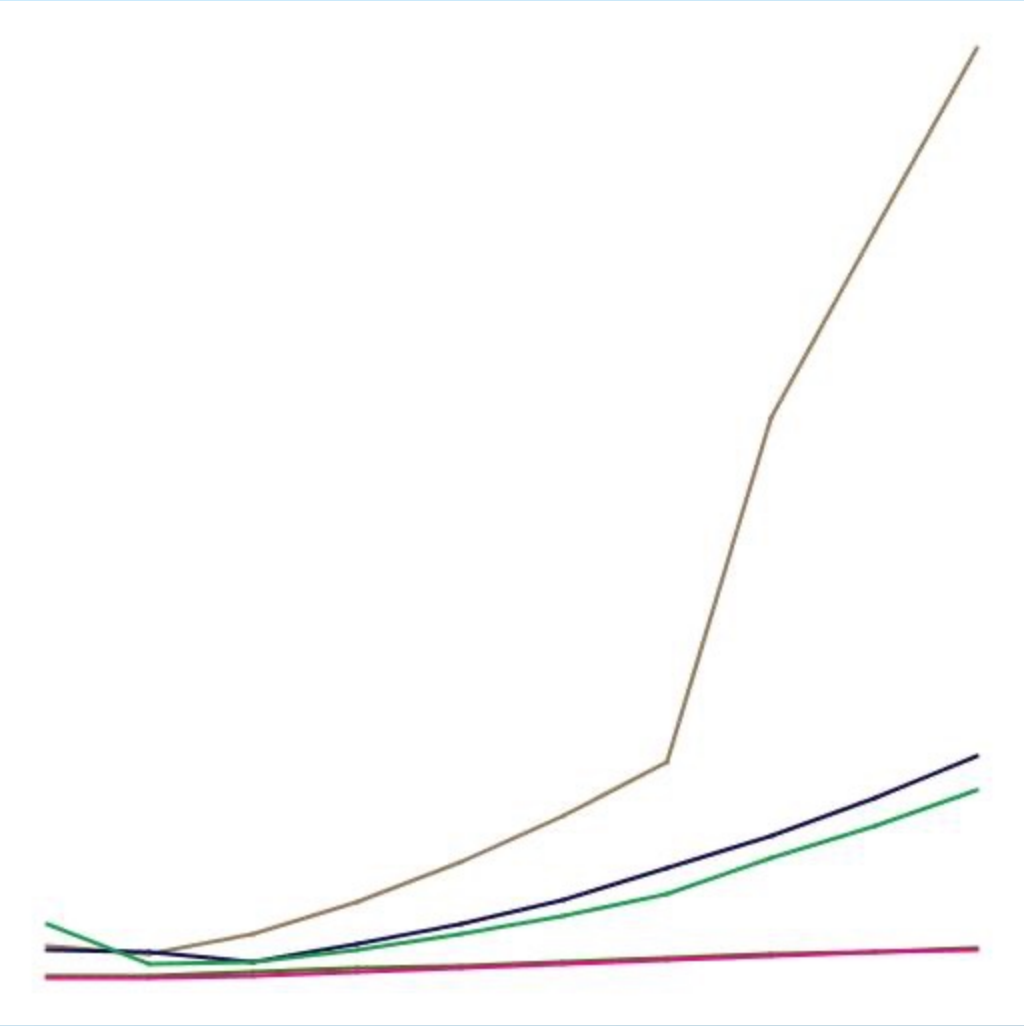
Merge sort - O(n log(n))

```
$ java SortLib
Size          Merge Sort          Quick Sort
100           12118.84            9355.34
200           14293.1             9578.58
300           18563.8             14482.22
400           27302.86            19691.54
500           28591.1             27571.04
600           38413.84            32502.92
700           46057.28            39917.5
800           49788.24            44032.44
900           51933.06            49646.7
1000          58949.88            55255.62
```



Merge sort – O(n log(n))

Quick sort – O(n log(n))



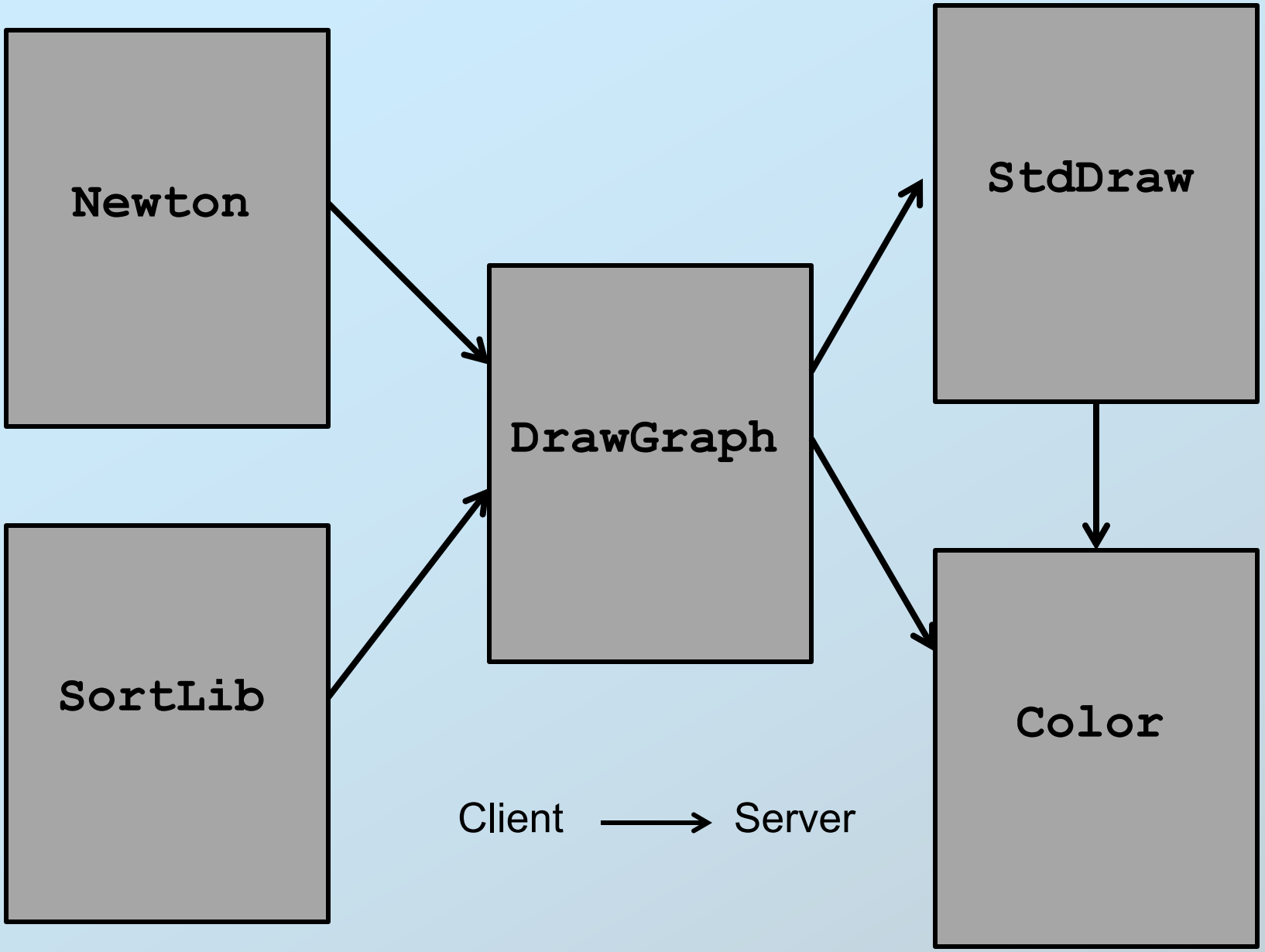
Bubble sort – $O(n^2)$

Insertion sort - $O(n^2)$

Selection sort - $O(n^2)$

Merge sort – O(n log(n))

Quick sort – O(n log(n))


Newton
StdDraw
DrawGraph
SortLib
Color
Client
Server

Σύνοψη

- Η έννοια της συνάρτησης είναι σημαντικότατη για το δομημένο προγραμματισμό
  - Αφαιρετικότητα προγράμματος
  - Αρθρωτές λύσεις
- Εξετάσθηκαν στατικές συναρτήσεις/μέθοδοι
- Διεπαφή – signature
  - Διαχωρισμός ανάμεσα στο «τι» και το «πώς»
  - Παράμετροι – return
- Κλήσεις – Ορίσματα – Binding
  - Πέρασμα δια τιμής
  - Τιμές που είναι διευθύνσεις (πίνακες, αντικείμενα)

Συνέχεια σύνοψης . . .

- Ροή Ελέγχου
- Εμβέλεια ονομασιών
- Function overloading
- Βιβλιοθήκες με επαναχρησιμοποιήσιμες συναρτήσεις
  - public – private
- Θα επανέλθουμε στις συναρτήσεις
  - Αναδρομικές συναρτήσεις
  - Μέθοδοι αναφοράς για αντικείμενα

Στο βιβλίο

- **Ενότητα 2.1**
- **Ενότητα 2.2**